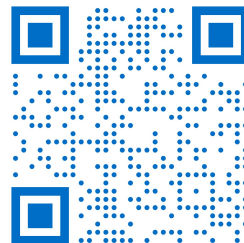


Андрей Державин,
Антон Шурыгин

Симуляция ЦП и ОС

Введение в Ruby





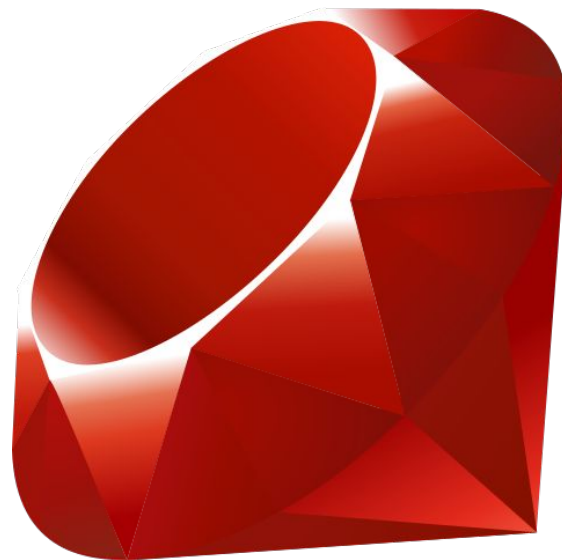
в симуляторах

- Execute и decode в одной функции
- CPU владеет RAM
- Адресация по 4 байта
- Sign Extension через `static_cast`
- Чтение программы через парсер

Введение

Ruby – это...

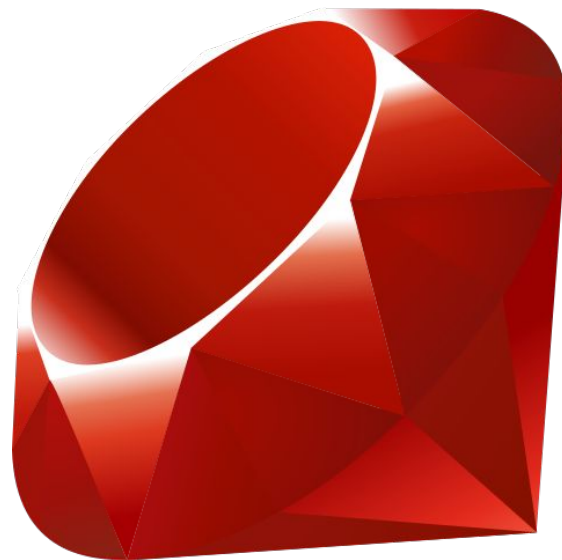
1. рефлексивный язык
2. высокоуровневый язык
3. динамический язык



Мотивация

Почему Ruby?

- Встроенная рефлексия
- Динамическая типизация
- Обширный ООП механизм
 - ВСЁ есть объект
 - Буквально всё
- Часто используется для небольших **DSL-решений**
- Метaprogramмирование



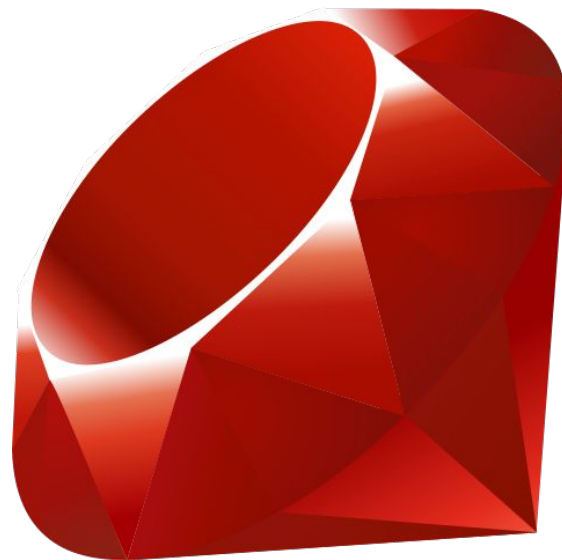
История языка

- Разработка началась в 1993 году Юкихиро Мацумото (Matz).
- Название навеяно языком Perl
- Первая стабильная версия вышла в 1995 году.
- В 1997 году появилось описание Ruby на английском языке
- Ruby был вдохновлен языками Perl, Smalltalk, Eiffel, Ada и Lisp.



Философия языка

- Основной принцип Ruby — принцип наименьшего удивления
 - Но не всегда
- Ruby также придерживается философии «счастья программиста»
- Ruby обладает мощной системой метапрограммирования



Теормин

- Необычные вызовы функций и return
- Открытые классы и миксины
- Символы
- Блоки
- Лямбды
- Метапрограммирование

Неявный return

- По умолчанию возвращается результат последнего выражения в блоке
- Какой вывод скрипта?

```
1  def foo(a, b)
2      if a > b
3          a - b
4      else
5          a + b
6      end
7  end
8
9  puts(foo(5, 1))
10 puts(foo(0, 2))
```

```
> ruby explicit_return.rb
4
2
```

True zero

```
1  if nil
2    puts "😂"
3  end
4  if false
5    puts "😍"
6  end
7  if 0
8    puts "😭"
9  end
```

```
> ruby if_nil.rb
```



- Ещё одна особенность – проверка на “nullptr”
- Какой смайлик будет напечатан?

```
1  puts "👍" if nil
2  puts "😞" if false
3  puts "🙌" if 0
```

```
> ruby if_nil_another.rb
```



Скобки для слабых

- Скобки опциональны
- Выражение вызова
матчится жадно
- Помните про читаемость!
- Что на экране?

```
1  def calc(a, b)
2      a + b
3  end
4
5  result = calc 11, 31
6  puts result
7  puts calc 10, 20
```

```
> ruby no_brackets.rb
42
30
```

Символы

- Строки – разные объекты в памяти

```
1 puts "user".object_id ≠ "user".object_id
```

```
> ruby strings.rb  
true
```

- Символы – один и тот же объект

```
1 puts :user.object_id == :user.object_id
```

```
> ruby symbols.rb  
true
```

Открытые классы

- В Ruby можно дополнить любой класс (включая встроенные) и новыми методами или изменить существующие.
 - Можем добавить функционал в класс String
 - Можем переосмыслить целочисленное сложение
- Что видим в консоли?

```
1  class String
2    def revdash
3      "---#{self.reverse}---"
4    end
5  end
6
7  puts "ybuR olleH".revdash
8
9  class Integer
10   def +(other)
11     self - other
12   end
13 end
14
15 puts 45 + 3
```

```
> ruby open_classes.rb
---Hello Ruby---
42
```

Примеси (англ. mixins)

- В Ruby нет множественного наследования
- Можно расширять функционал классов с помощью модулей.
- Модули – контейнеры для методов, констант и других модулей.
 - **include** – добавляем методы модуля как методы экземпляра.
 - **extend** – добавляем методы модуля как методы класса.
- Какой вывод в консоль?

```
1 module Loggable
2   def log(message)
3     puts "[#{Time.now}] #{message}"
4   end
5 end
6
7 class User
8   include Loggable
9   def save
10    log "Saving user ..."
11  end
12 end
13
14 class Configuration
15   extend Loggable
16   def self.load
17    log "Loading config ..."
18  end
19 end
20
21 user = User.new
22 user.save
23 Configuration.load
```

```
> ruby mixins.rb
[2025-09-24 22:50:41 +0300] Saving user...
[2025-09-24 22:50:41 +0300] Loading config...
```

Блоки

- Анонимные функции
- Можно передавать как аргументы
- Можно исполнять в разных контекстах
- Часто используются при работе с контейнерами

```
1 numbers = [1, 2, 3, 4, 5]
2 numbers.each do |number|
3   puts number
4 end
```

```
1 5.times do
2   puts "Hi"
3 end
4
5 10.times { puts "Hello" }
```

```
1 def say(&block)
2   block.call("Ruby")
3 end
4
5 say do |name|
6   puts "Hello, #{name}!"
7 end
```

Proc

- Объект, хранящий блок

```
1 def multiplier(n)
2   Proc.new { |x| x * n }
3 end
4
5 double = multiplier(2)
6 puts double.call(5)
```

```
> ruby proc_1.rb
10
```

```
1 doubler = proc { |x| x * 2 }
2 puts doubler.call(2)
```

```
1 p1 = Proc.new { "hello" }
2 p2 = Proc.new { |x| "hello #{x}" }
3
4 puts p1.call("world")
5 puts p2.call
```

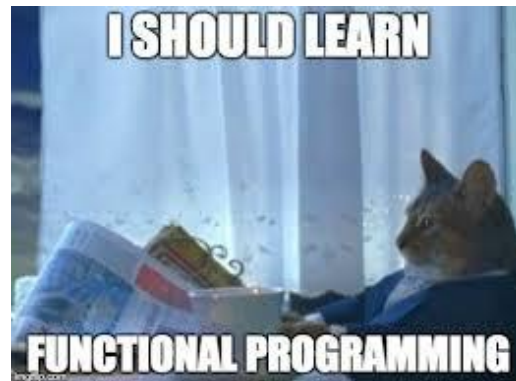
```
> ruby proc_2.rb
hello
hello
```

Lambdas

- Мало функциональных объектов?
- По факту тот же Proc
- Проверяет аргументы
- Больше похож на функцию

```
1 def test_return
2   proc = Proc.new { return "proc" }
3   lambda = → { return "lambda" }
4
5   puts lambda.call
6   proc.call
7   "return"
8 end
9
10 puts test_return
```

```
> ruby lambda_1.rb
lambda
proc
```



```
1 p = Proc.new { |a, b| [a, b] }
2 l = →(a, b) { [a, b] }
3
4 puts p.call(1).inspect
5 puts l.call(1).inspect
```

```
> ruby lambda_2.rb
[1, nil]
lambda_2.rb:2:in 'block in <main>': wrong number of arguments (given 1, expected 2) (ArgumentError)
from lambda_2.rb:4:in '<main>'
```

Open-closed principle?

- `instance_eval` - позволяет исполнить блок в контексте экземпляра класса
- `class_eval` - позволяет исполнить блок в контексте класса
- Крайне полезно в DSL
- Что на экране?

```
1 class Counter
2   def initialize
3     @count = 0
4   end
5 end
6
7 counter1 = Counter.new
8 counter2 = Counter.new
9
10 counter1.instance_eval do
11   def increment
12     @count += 1
13   end
14 end
15
16 Counter.class_eval do
17   def decrement
18     @count -= 1
19   end
20 end
21
22 puts counter1.increment
23 puts counter2.increment
24
25 puts counter1.decrement
26 puts counter2.decrement
```

Open-closed principle?

- `instance_eval` - позволяет исполнить блок в контексте экземпляра класса
- `class_eval` - позволяет исполнить блок в контексте класса
- Крайне полезно в DSL
- Что на экране?
 - Окей, уберем ошибку

```
1 class Counter
2   def initialize
3     @count = 0
4   end
5 end
6
7 counter1 = Counter.new
8 counter2 = Counter.new
9
10 counter1.instance_eval do
11   def increment
12     @count += 1
13   end
14 end
15
16 Counter.class_eval do
17   def decrement
18     @count -= 1
19   end
20 end
21
22 puts counter1.increment
23 # puts counter2.increment
24
25 puts counter1.decrement
26 puts counter2.decrement
```

```
> ruby instance_eval.rb
1
0
-1
```

Нет метода

- Вызов неизвестных методов
- Полезно для отладки, логирования или создания заглушек

```
class Something
  def method_missing(name, *args)
    puts "Called #{name} with args #{args}"
  end
end

a = Something.new
a.foo(1, 2)
a.bar
```

```
> ruby method_missing.rb
Called foo with args [1, 2]
Called bar with args []
```

Есть метод

- Можем создавать методы динамически, когда их имена известны во время выполнения.
- Удобно, когда список команд или атрибутов задан данными.

```
class Robot
  [:walk, :jump].each do |action|
    define_method(action) { puts "Robot can #{action}" }
  end
end

r = Robot.new
r.walk
r.jump
```

```
> ruby define_method.rb
Robot can walk
Robot can jump
```

Отправка метода

- Вызываем метод по имени.
- Имя может быть вычислено во время выполнения
- Удобно для динамического диспетчерирования
- Есть `public_send`

```
class Greeter
  def hello(name) = "Hello, #{name}!"
end

g = Greeter.new
m = :hello
puts g.send(m, "Ruby")
```

Свобода не бесконечна

- Можно перегружать операторы
- Нельзя перегружать = и подобные
 - Есть предположения почему?
- Логические операторы тоже не перегружаем

В первый раз – первый класс

- Создадим наш первый класс на Ruby
- Рассмотрим класс двумерной точки Point.
 - Пока что класс довольно бесполезный

```
1  class Point
2      @x = 0
3      @y = 0
4  end
```

В первый раз – первый класс

- Создадим наш первый класс на Ruby
- Рассмотрим класс двумерной точки Point.
- Определим конструктор
 - Переменные экземпляра класса в Ruby по умолчанию являются приватными

```
1 class Point
2   def initialize(x, y)
3     @x = x # but now it's private
4     @y = y
5   end
6 end
```

В первый раз – первый класс

- Создадим наш первый класс на Ruby
- Рассмотрим класс двумерной точки Point.
- Определим конструктор
- Добавим атрибутов

```
1  class Point
2    attr_reader :x, :y # getter
3    attr_writer :x, :y # setter
4    def initialize(x, y)
5      @x = x
6      @y = y
7    end
8  end
```

В первый раз – первый класс

- Создадим наш первый класс на Ruby
- Рассмотрим класс двумерной точки Point.
- Определим конструктор
- Добавим атрибутов
- Перегрузим оператор +
- Создадим объект класса
 - @x – прямой доступ к x
 - self.y – ссылка на текущий объект + вызов getter'a y

```
1  class Point
2    attr_accessor :x, :y
3    def initialize(x, y)
4      @x = x
5      @y = y
6    end
7
8    def to_s
9      "Point(#{@x}, #{@y})"
10   end
11
12   def +(other)
13     Point.new(@x + other.x,
14               @y + other.y)
15   end
16 end
17
18 p1 = Point.new(3, 4)
19 p2 = Point.new(5, 6).freeze
```

В первый раз – первый класс

- Создадим наш первый класс на Ruby
- Рассмотрим класс двумерной точки Point.
- Определим конструктор
- Добавим атрибутов
- Перегрузим оператор +
- Создадим объект класса
- Наследование
 - `super(x,y)` – вызов конструктора базового класса Point

```
1  require_relative 'point'
2
3  class Point3D < Point
4    attr_accessor :z
5
6    def initialize(x, y, z)
7      super(x, y)
8      @z = z
9    end
10 end
11
12 point3d = Point3D.new(1, 2, 3)
```

Прим.: реализовано в обучающих целях, не является примером хорошего дизайна

В первый раз – первый класс

- Создадим наш первый класс на Ruby
- Рассмотрим класс двумерной точки Point.
- Определим конструктор
- Добавим атрибутов
- Перегрузим оператор +
- Создадим объект класса
- Наследование
 - @@total – переменная класса, **разделяется** между всеми экземплярами класса и **наследуются**

```
1  require_relative 'point'
2
3  class Point3D < Point
4    attr_accessor :z
5    @@total = 0
6    def initialize(x, y, z)
7      super(x, y)
8      @z = z
9    end
10 end
11
12 point3d = Point3D.new(1, 2, 3)
```

*Прим.: реализовано в обучающих
целях, не является примером
хорошего дизайна*

Как использовать?

- apt (Debian or Ubuntu)

```
sudo apt-get install ruby-full
```

- macOS

```
brew install ruby
```

- Для других платформ – читать [здесь](#)
- Интерактивный интерпретатор

Домашнее задание №2

- Написать на Ruby программу, которая позволяет описывать машинный код в виде, похожем на ассемблер.
- Каждая строка этого 'ассемблера' является вызовом метода Ruby.
- В результате работы микро ассемблера создаётся файл с бинарными кодами инструкций
- Этот файл читается вашим интерпретатором и выполняется
- Задача со *: поддержка меток

Домашнее задание №2

```
1 # Вот что ожидается на микро-ассемблере:  
2 prog do  
3     mov r1, 42  
4     add r2, 1  
5     hlt  
6 end
```

Ссылки

- [Ruby](#)
- [Ruby in Twenty Minutes](#)
- [Openness Modules & Class](#)
- [Module: mixins and namespace](#)
- [Ruby From Other Languages](#)
- [Installing Ruby](#)
- <https://try.ruby-lang.org/>
- [Как быстро и просто написать DSL на Ruby](#)